

PARTE 3: INTERACTIVIDAD Y PROGRAMACIÓN CON JAVASCRIPT

Unidad 4: JavaScript + DOM

Fecha de entrega: Hasta el 01.07.2026 (23.30 hs.).

A. Consignas Técnicas (JavaScript)

En esta etapa deberán incorporar funcionalidades interactivas al proyecto desarrollado en las fases anteriores utilizando JavaScript.

El objetivo es mejorar la experiencia del usuario mediante eventos, manipulación del DOM y programación estructurada.

1. Archivo JavaScript externo

- Crear un archivo script.js.
- Vincular correctamente el archivo desde index.html.
- No se permite escribir código JavaScript directamente dentro de las etiquetas HTML (onclick, onchange, etc.).

2. Uso de funciones

Implementar al menos:

- 3 funciones diferentes.
- Cada función debe tener una responsabilidad clara.
- Deben ser invocadas desde eventos o desde otras funciones.

Ejemplos

- Mostrar información.
- Agregar elementos dinámicamente.
- Validar formularios.
- Filtrar contenido.
- Calcular datos.

3. Eventos

Implementar al menos **3 eventos diferentes**.

Ejemplos

- click
- input
- change
- submit
- mouseover
- keyup

Los eventos deben generar una acción visible para el usuario.

4. Uso de Arrays

Incorporar al menos un array para almacenar información utilizada por el proyecto.

Ejemplos

Manual de Supervivencia

```
const consejos = [  
  "Llevar agua",  
  "Revisar el clima",  
  "Cargar batería"  
];
```

Portfolio

```
const proyectos = [  
  "Sistema de turnos",  
  "Página institucional",  
  "Calculadora web"  
];
```

El contenido del array debe utilizarse para mostrar información en la página.

5. Manipulación del DOM

Implementar al menos **dos acciones sobre el DOM**.

Ejemplos

- Crear elementos.
- Modificar texto.
- Cambiar estilos.
- Mostrar u ocultar contenido.
- Agregar elementos a listas.

6. Validación y manejo de errores

El formulario de contacto desarrollado en la primera etapa deberá incluir validaciones.

Requisitos mínimos

- Verificar que los campos obligatorios no estén vacíos.
- Mostrar mensajes de error al usuario.
- Además, utilizar al menos 1:

```
try {  
  ...  
}  
catch(error) {  
  ...  
}
```

para controlar una situación de error o validación.

7. Funcionalidad principal

El proyecto debe incorporar al menos **una funcionalidad interactiva relevante** relacionada con la temática elegida.

Ejemplos

Manual de Supervivencia

- Buscador de consejos.
- Lista de elementos necesarios.
- Generador de recomendaciones.

Portfolio

- Filtro de proyectos.
- Buscador de tecnologías.
- Galería interactiva.
- Sección de proyectos dinámica.

B. Consignas de Git y GitHub

1. Continuidad del repositorio

- Utilizar el mismo repositorio empleado en las fases anteriores.

2. Historial de commits

Se requiere un mínimo de: **4 commits nuevos**

Ejemplos

feat: agregar validación de formulario

feat: implementar buscador de proyectos

fix: corregir manejo de errores

refactor: reorganizar funciones

3. README.md

Actualizar nuevamente el archivo README incorporando:

- Descripción final del proyecto.
- Tecnologías utilizadas (HTML, CSS y JavaScript).
- Capturas actualizadas (opcional).
- Funcionalidades implementadas.
- Enlace al repositorio.

Tip

"El objetivo no es agregar muchas funcionalidades, sino demostrar que comprendés cómo JavaScript permite que una página web deje de ser estática y pueda interactuar con el usuario."

C. CRITERIOS DE EVALUACIÓN (RÚBRICA)

Criterio	0 puntos	1 punto	2 puntos
Uso correcto de funciones	No utiliza funciones o presenta errores importantes.	Utiliza funciones de forma básica o poco organizada.	Implementa funciones correctamente, con responsabilidades claras y reutilización adecuada.
Implementación de eventos	No utiliza eventos o no funcionan correctamente.	Utiliza eventos básicos con funcionamiento parcial.	Implementa correctamente distintos eventos que generan interacciones significativas.
Utilización de arrays	No utiliza arrays o los utiliza incorrectamente.	Utiliza arrays de forma limitada o poco relevante.	Utiliza arrays correctamente para almacenar y gestionar información del proyecto.
Manipulación del DOM	No modifica el DOM o presenta errores importantes.	Realiza modificaciones básicas o limitadas.	Manipula el DOM correctamente mediante creación, modificación o eliminación de elementos.
Validaciones y manejo de excepciones	No implementa validaciones o manejo de errores.	Implementa validaciones básicas o manejo parcial de excepciones.	Realiza validaciones adecuadas y utiliza correctamente estructuras de manejo de excepciones.
Calidad de la funcionalidad implementada	La funcionalidad es inexistente o no cumple su objetivo.	La funcionalidad cumple parcialmente su objetivo.	La funcionalidad es útil, coherente con la temática y mejora la experiencia del usuario.
Orden y limpieza del código	Código desordenado o difícil de interpretar.	Organización mejorable o inconsistente.	Código bien indentado, organizado, comentado y fácil de leer.
Historial de commits y actualización del README	Historial insuficiente o README sin actualizar.	Pocos commits o README incompleto.	4 o más commits descriptivos y README correctamente actualizado.
Integración con las fases anteriores	El proyecto presenta inconsistencias o pérdida de funcionalidades previas.	Integra parcialmente HTML, CSS y JavaScript.	Integra correctamente las tres etapas manteniendo coherencia visual y funcional.